O que foi feito na pesquisa

Previsão de sequências do vírus influenza: resumo do trabalho, do código e de como executar

28 de outubro de 2026

Este documento explica, de forma direta, tudo o que foi feito sobre o código da pesquisa original: o que foi corrigido, o que foi criado, como o projeto está organizado e como reproduzir os resultados. A análise completa dos resultados está em `relatorios/relatório e análise.pdf`.

1. Ponto de partida

O código recebido era o de um TCC que tentava prever a sequência dos genes HA e NA do influenza (cepas H1N1 e H3N2) na temporada seguinte. Cada sequência era transformada em números (A=1, T=2, G=3, C=4, com cada 15 dígitos formando um número) e quatro modelos do scikit-learn aprendiam a ir da sequência antiga para a nova: Árvore de Decisão, Floresta Aleatória, Árvores Extras e Naive Bayes. A nota usada, chamada de "acurácia", era o R^2 multiplicado por 100.

Ao tentar rodar e estender esse código, encontramos estes problemas:

Problema	Consequência
Código escrito para Python 3.8 e bibliotecas antigas	O Naive Bayes não roda mais no scikit-learn atual.
`data_test[i]` no lugar de `data_test[j]`	Todas as sequências-alvo eram iguais, e os resultados saíam inflados (até 100%).
Sequências de H1N1 começando em pontos diferentes	A posição *i* não era o mesmo ponto do gene em cada par.
Naive Bayes dividido de outra forma	Seus resultados não eram comparáveis com os dos outros modelos.
Conjunto escolhido comentando linhas do código	Fácil rodar o experimento errado.

2. O que foi feito, passo a passo

2.1 Atualização do código

O código foi atualizado para Python 3.12 e para as versões atuais das bibliotecas (scikit-learn 1.9, NumPy 2.5, Biopython 1.88). O Naive Bayes voltou a funcionar convertendo os números de 15 dígitos em inteiros, sem perda de informação. Sequências de tamanhos diferentes passaram a ser completadas com zeros, e o conjunto e os modelos agora são escolhidos pela linha de comando (`--dataset`, `--models`).

2.2 Correção do erro no código original

No `main.py` original, o laço que lia os alvos percorria `j` mas usava `i`, que tinha ficado parado no último valor do laço anterior. Assim, todos os alvos eram cópias da última sequência do arquivo, e os modelos só precisavam devolver sempre a mesma sequência. Rodando o código com o erro, reproduzimos exatamente os valores da Figura 12 da pesquisa original (100,00%, 91,23% e 100,00%).

2.3 Alinhamento das sequências

Em H1N1, algumas sequências trazem bases antes do códon de início do gene (ATG) e outras não. A opção `--align` corta cada sequência no ATG que dá a maior região codificante (função `trim_to_start_codon` em `dataset.py`). Sem isso, comparar posição por posição não faz sentido.

2.4 Avaliação igual para todos os modelos

- **Mesmas divisões:** todos os modelos usam a validação cruzada com 2 partes e a divisão 70% treino / 30% teste (`random_state=50`) da pesquisa original.
- **Modelo de referência:** "Cópia da temporada anterior", que só repete a sequência de entrada. Qualquer modelo útil precisa ser melhor que ele.
- **Novas métricas:** acurácia por nucleotídeo (quantas letras o modelo acertou) e acurácia nas mutações (acerto só nas posições que mudaram). As métricas antigas (R^2, MAE, MSE) continuam sendo calculadas.
- **Mesma régua:** as previsões das redes neurais são convertidas para a codificação de 15 dígitos, e as dos modelos clássicos são convertidas de volta em letras, para que todos recebam todas as notas.

2.5 Novos modelos

Foram criados três modelos de deep learning, em PyTorch. Todos recebem a sequência antiga e dão, para cada posição do gene, uma pontuação para A, T, G e C:

- **CNN:** rede convolucional que lê letra por letra, com 4 blocos que enxergam cada vez mais longe.
- **Transformer:** lê o gene em códons (palavras de 3 letras); cada códon pode olhar para todos os outros.
- **BiLSTM:** o modelo clássico de processamento de linguagem natural; lê os códons nos dois sentidos.

Os três compartilham duas ideias de processamento de linguagem natural: a **tokenização por códons** e um **mecanismo de cópia**, um atalho que favorece manter a letra de entrada, já que cerca de 99% do gene não muda entre temporadas. Treino: otimizador AdamW, 200 épocas, lotes de 8 pares.

2.6 Repetição com várias sementes

Modelos com componente aleatório podem ganhar ou perder por sorte. Por isso, cada modelo foi executado com 5 sementes aleatórias (opção `--seeds`), e os resultados são apresentados como média ± desvio padrão. Isso mostrou que algumas vitórias dos modelos clássicos na primeira execução eram sorte.

2.7 Verificação dos dados maiores

Também conferimos os arquivos maiores de sequências que acompanhavam a pesquisa original (com milhares de sequências de H3N2). Só os dados de H3N2 de 2021 e 2022 seriam aproveitáveis num experimento maior, depois de remover sequências repetidas e parciais. Esse experimento não foi feito, e esses arquivos não fazem parte desta versão do projeto.

3. Organização do projeto

```
flu-prediction-master/
├── main.py                 programa principal: escolhe conjunto e modelos, roda e grava resultados
├── dataset.py              leitura dos FASTA, alinhamento e preparação dos dados
├── evaluation.py           divisões de treino/teste, modelo de referência e métricas
├── encoding_data.py        codificação da pesquisa original (A=1, T=2, G=3, C=4, blocos de 15)
├── prediction_model/
│   ├── decisionTree.py     modelos clássicos da pesquisa original
│   ├── randomForest.py
│   ├── extraTrees.py
│   ├── naiveBayes.py       Naive Bayes corrigido para o scikit-learn atual
│   └── deep/               redes neurais novas
│       ├── layers.py       tokenização por códons e camada de saída com cópia
│       ├── cnn.py          CNN
│       ├── transformer.py  Transformer
│       ├── lstm.py         BiLSTM
│       └── trainer.py      treino e previsão das redes neurais
├── results/                resultados em CSV e saída completa de cada execução
├── relatorios/             relatórios em PDF
├── flu-data/               sequências em FASTA usadas nos experimentos (da pesquisa original)
├── requirements.txt        bibliotecas necessárias
├── LICENSE                 licença MIT da pesquisa original
└── README.md               instruções de instalação e uso
```

4. Como executar

Instalação (uma vez), dentro da pasta do projeto:

```
python -m venv .venv
.venv\Scripts\activate
pip install torch --index-url https://download.pytorch.org/whl/cu126   # só com placa NVIDIA
pip install -r requirements.txt
```

Experimentos:

```
python main.py --dataset all --align --seeds 5      # experimento principal (cerca de 1h40 numa MX570)
python main.py --dataset all --align                # execução única, mais rápida (cerca de 20 min)
python main.py --dataset h1n1-ha --models classical # só os modelos da pesquisa original
```

5. Resultados em resumo

Média dos quatro conjuntos e das 5 sementes, no conjunto de teste, com as sequências alinhadas:

Modelo	Tipo	Acurácia por nucleotídeo	Acurácia nas mutações
Cópia da temporada anterior	Referência	98,61% ± 0,00	0,00% ± 0,00
Árvore de Decisão	Clássico	91,40% ± 0,35	41,92% ± 0,79
Floresta Aleatória	Clássico	84,05% ± 0,77	32,86% ± 0,94
Árvores Extras	Clássico	89,76% ± 0,67	37,48% ± 0,43
Naive Bayes	Clássico	82,22% ± 0,00	39,80% ± 0,00
CNN	Deep learning	98,76% ± 0,02	38,37% ± 0,35
Transformer	Deep learning	99,26% ± 0,01	34,11% ± 0,34
BiLSTM	Deep learning	99,19% ± 0,01	34,58% ± 0,14

Os modelos de deep learning foram os mais precisos e os mais estáveis, mas ficaram pouco acima de simplesmente repetir a sequência da temporada anterior. Nas posições que sofreram mutação, os modelos clássicos acertam uma parcela parecida ou maior. Com 16 a 35 pares de treino por conjunto, não é possível afirmar que uma abordagem seja claramente superior.

6. Observações

- **Pequenas variações na placa de vídeo:** as redes neurais podem dar resultados um pouco diferentes (em torno de 0,05 ponto) mesmo com a mesma semente, porque a placa de vídeo usa cálculos que não são perfeitamente repetíveis. Os modelos clássicos dão sempre o mesmo resultado.
- **Código de saída do BiLSTM:** no Windows, depois que o BiLSTM roda na placa de vídeo, o Python pode terminar com o código -1073740791. Isso acontece no driver da NVIDIA, depois que todos os resultados já foram gravados, e não afeta os números.
- **Créditos:** o código e os dados de partida são do TCC original, que por sua vez se inspirou no trabalho de Koodli (github.com/RK900/Flu-Prediction).